



Unidad IV

Clase 7



Implementaciones dinámicas

PENSAR EN COSTOS

*Desarrollar pensamiento analítico para
justificar decisiones de diseño.*



¿Por qué pensar en costos? Eficiencia temporal y espacial

Especificación

Describe qué hace la estructura y qué comportamiento debe cumplir, ignorando los detalles internos.

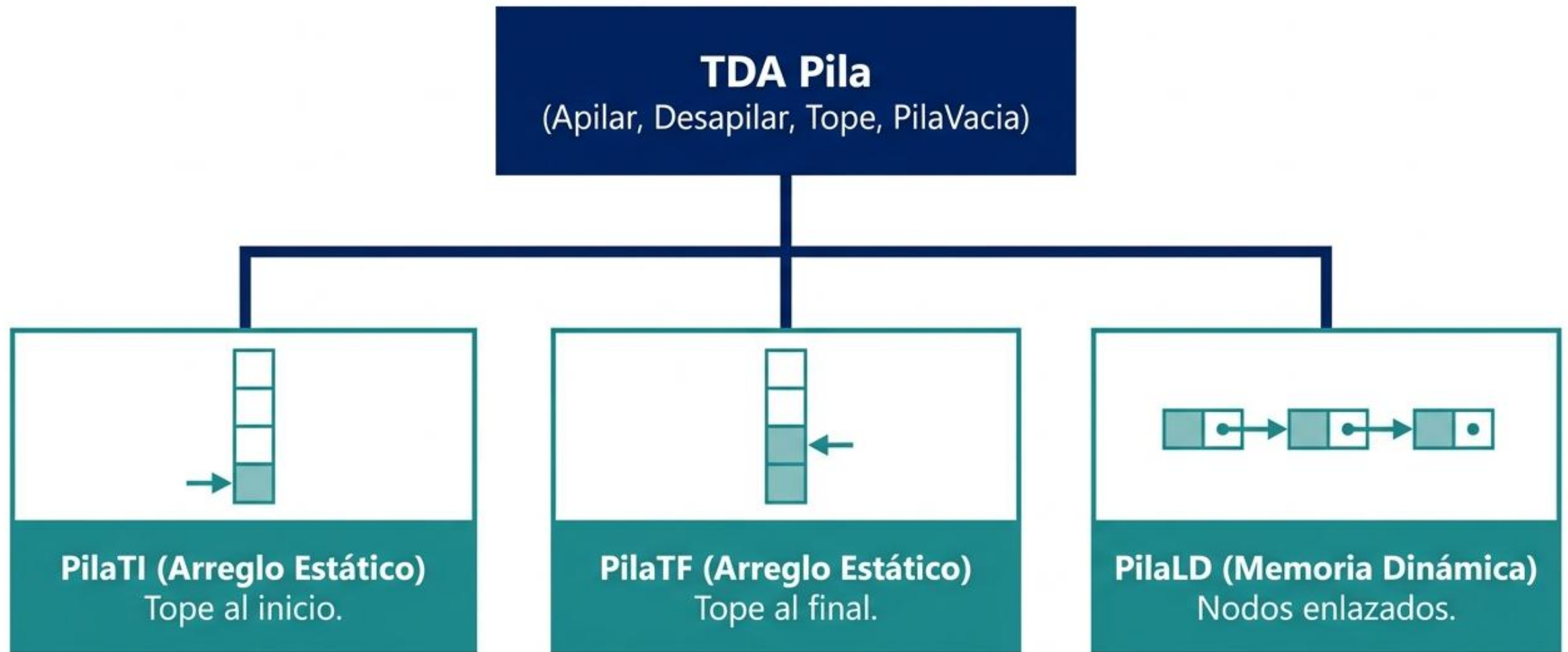
El TDA define el comportamiento. La implementación define CÓMO se resuelve y CUÁNTO cuesta.



Resolución técnica

Define cómo se almacenan los datos en memoria y cómo se resuelve internamente cada operación.

Especificación vs. Implementación: El Caso de la Pila



Dos implementaciones pueden respetar el mismo TDA y tener costos distintos.

Criterios de Complejidad Temporal



Costo Constante (C)

El tiempo de ejecución es independiente de la cantidad de elementos.

Origen técnico:

Acceso directo (índices de arreglos) o cambios de referencia (punteros en listas).



Costo Lineal (L)

El trabajo crece en proporción a la cantidad de elementos (n).

Origen técnico:

Recorridos secuenciales (búsquedas) o corrimientos elementos en arreglos.



Costo Polinómico (P)

El trabajo crece desproporcionadamente (cuadrático, cúbico).

Origen técnico:

Recorridos combinados o bucles anidados.

Análisis PilaTDA: Comparación de Implementaciones

Operación	PilaTI (Inicio)	PilaTF (Final)	PilaLD (Nodos)
InicializarPila	C	C	C
Apilar	L	C	C
Desapilar	L	C	C
Tope	C	C	C
PilaVacía	C	C	C

Conclusión

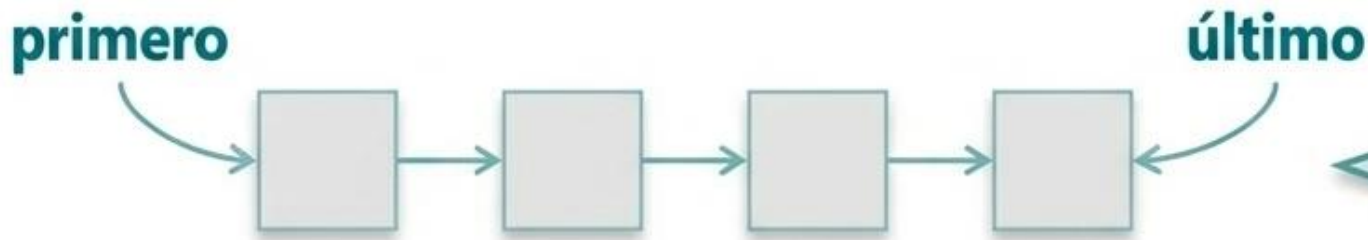
La dinámica (PilaLD) mejora notablemente frente a PilaTI al evitar los corrimientos obligatorios.

Sin embargo, **no mejora temporalmente frente a PilaTF**, ya que esta última logra eficiencia óptima (costo constante) utilizando acceso directo por índice.

Análisis ColaTDA: El problema del corrimiento

Operación	ColaPU (Último)	ColaPI (Inicio)	ColaLD (Dinámica)
InicializarCola	C	C	C
Acolar	L	C	C
Desacolar	C	L	C
Primero	C	C	C
ColaVacía	C	C	C

Detalle Estructural



La ColaLD permite **Acolar** y **Desacolar** en costo constante (C) porque utiliza dos referencias. Elude los corrimientos penalizantes de las versiones estáticas.

Análisis ColaPrioridadTDA: El costo del orden

Operación	DA (Doble Arreglo)	AO (Arreglo Objetos)	LD (Lista Dinámica)
InicializarCola	C	C	C
AcolarPrioridad	L	L	L
Desacolar	C	C	C
Primero	C	C	C
ColaVacia	C	C	C
Prioridad	C	C	C

¿Por qué AcolarPrioridad sigue siendo Lineal?

Mantener el orden según la prioridad exige encontrar la posición correcta antes de insertar.

Versiones Estáticas (DA, AO): El costo (L) proviene del recorrido secuencial más el **corrimiento** físico de los elementos en el arreglo.

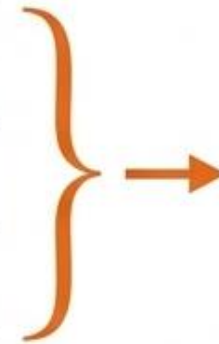
Versión Dinámica (LD): El costo (L) proviene exclusivamente del **recorrido secuencial** por los nodos para encontrar dónde modificar las referencias.

Análisis ConjuntoTDA: Búsquedas sin orden

Sin orden interno

Sin elementos repetidos

Operación	ConjuntoA (Arreglo)	ConjuntoLD (Nodos)
InicializarConjunto	C	C
Agregar	L	L
Sacar	L	L
Pertenece	L	L
Elegir	C	C
ConjuntoVacio	C	C



El cuello de botella es la validación de unicidad o existencia.

Conclusión: Pasar de un arreglo a nodos dinámicos optimiza la memoria, pero **no mejora el costo temporal principal**. La búsqueda secuencial sigue siendo obligatoria.

Diccionarios (Simple y Múltiple): El peso de la clave

Top Panel

Diccionario Simple (DicSimpleA & DicSimpleL)



En los diccionarios, la eficiencia está dominada por la búsqueda secuencial de la clave (y del valor, en los múltiples). La estructura dinámica aporta flexibilidad espacial para crecer bajo demanda, pero el costo temporal no cambia.

Bottom Panel

Diccionario Múltiple (DicMultipleA & DicMultipleL)



Criterios de Selección: ¿Qué implementación elegir?



**Cola con alto
flujo (I/O)**

**ColaLD
(Dinámica)**

Evita los corrimientos penalizantes que ocurren constantemente al acolar o desacolar en versiones estáticas.



**Pila simple y
predecible**

**PilaTF
(Estática)**

Ya ofrece costo Constante (C) general; es fácil de programar y muy eficiente si se conoce de antemano el tamaño máximo (tope).



**Estructuras de
búsqueda**

**Empate
Temporal**

Conjuntos y Diccionarios siguen siendo Lineales (L) en ambos casos.
Elegir dinámica si el tamaño variará mucho.

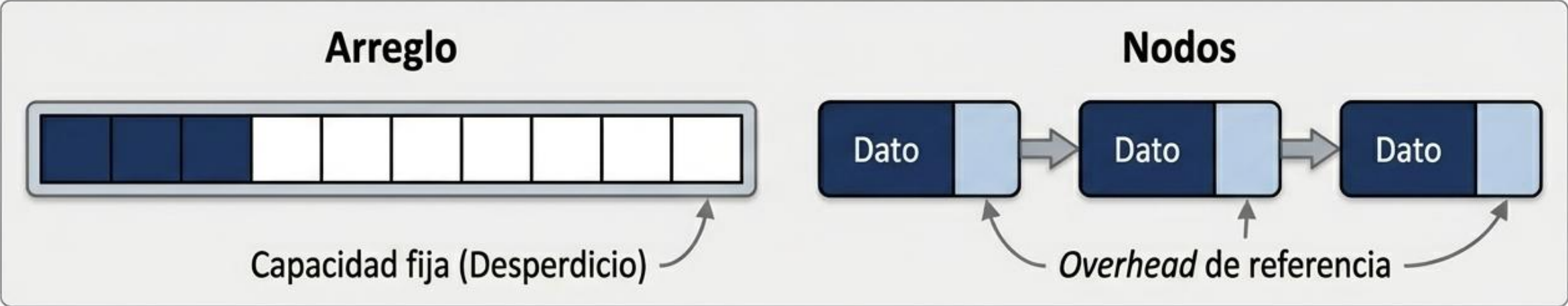


**Evitar el
desperdicio**

**Siempre
Dinámica**

Usar listas enlazadas cuando la cantidad máxima de elementos es incierta o altamente variable.

Costo Espacial: Arreglos estáticos vs. Nodos dinámicos



Criterio	Arreglos (Estático)	Nodos (Dinámico)
Reserva de memoria	Reserva total previa	Demanda "On the fly"
Riesgo principal	Desperdicio por lugares vacíos o desbordamiento	Mayor consumo por elemento debido a las referencias
Ventaja principal	Acceso contiguo en memoria	Crece y decrece según la necesidad exacta
Relación tamaño	Capacidad desvinculada del uso real	Crece 1 a 1 con los elementos (n)

Síntesis Final: Razonar, no memorizar

Costo Constante (C)

Las operaciones son instantáneas porque **accedés directamente** (por índice) o simplemente **reassignás referencias** (punteros). No hay que moverse.

Costo Lineal (L)

El costo aumenta con los datos porque te ves obligado a **recorrer secuencialmente** (buscar claves/posiciones) o **desplazar datos en bloque** (corrimiento).

Costo Polinómico (P)

El sistema se satura porque **combinás o anidás recorridos** para procesar un solo evento.

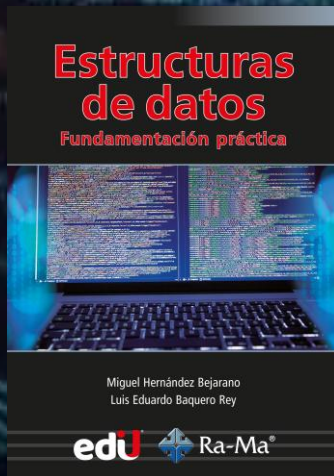
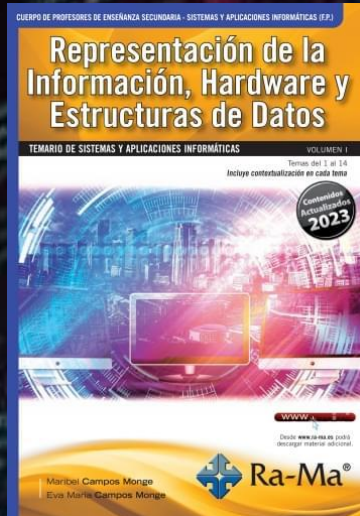
**Un TDA define el comportamiento;
la implementación define el costo.**

Actividad

Comparación de costos en implementaciones estáticas y dinámicas

Una empresa necesita implementar un sistema simple de gestión de solicitudes internas. El sistema debe permitir registrar solicitudes comunes, atender solicitudes urgentes, evitar códigos duplicados, asociar empleados con las solicitudes que cargaron y deshacer la última acción realizada.

El objetivo de la actividad es analizar qué TDA conviene utilizar en cada necesidad, comparar implementaciones estáticas y dinámicas, identificar los costos temporales principales y justificar la decisión de diseño.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 8

Pensar jerárquicamente: introducción a Árboles y ABB

Concepto de árbol.

Recursividad.

Árbol Binario de Búsqueda (ABB).

Recorridos.